Machine Learning & Deep Learning Cheat Sheet

Framework-neutral quick reference: choosers, recipe, debugging and deployment (October 2026)

Learning types

Type	Signal / examples
Supervised	labels y; regression / classification
Unsupervised	no labels; clustering, density, dim. reduction
Self-supervised	labels from the data (next token, masked patch)
Semi-supervised	few labels + many unlabeled
Reinforcement	reward from an environment; policy / value
Transfer	pretrained model, fine-tune on small data

Loss chooser

Task	Loss
Regression	MSE (default), MAE / Huber if outliers
Binary	BCE with logits; sigmoid only at inference
Multiclass	cross-entropy on raw logits (no softmax)
Multilabel	BCE with logits per label
Imbalanced	class weights, focal loss, resampling
Ranking / embed.	contrastive, triplet, InfoNCE
Generative	NLL / ELBO, denoising MSE (diffusion)

Metric chooser

Case	Metric
Balanced classes	accuracy, macro-F1
Imbalanced	PR-AUC, recall@precision, F1
Ranking	ROC-AUC, NDCG, MAP
Regression	RMSE, MAE, R^2, MAPE
Calibration	log loss, Brier, ECE
Language	perplexity; task-specific evals

Pick the metric before the model; tune on validation, report once on test.

Activations and output layer

Hidden (MLP/CNN)	ReLU; GELU / SiLU in transformers
Output regression	linear (none)
Output binary	sigmoid (inside the loss)
Output multiclass	softmax (inside the loss)
Gates (RNN)	sigmoid, tanh

Optimiser defaults

Adam / AdamW	lr 1e-3 (CNN/MLP), 1e-4 to 3e-4 (transformers); AdamW wd 0.01-0.1
SGD + momentum	lr 0.1, momentum 0.9, cosine / step decay (vision)
Warmup	linear 1-5 % of steps, then cosine decay
Clipping	grad-norm 1.0 for RNN / transformers
Batch size	largest that fits; scale lr with it

Training recipe

- Frame the problem, baseline (majority / linear / GBM)
- Split train / val / test (group / time aware); no leakage
- Normalise inputs with train statistics only
- Overfit one small batch first (loss to ~0)
- Pick loss + metric, optimiser, lr, schedule
- Train, log curves; early stop on val metric
- Tune lr first, then batch, width, regularisation
- Fix seeds, log config and data version
- Evaluate once on test; check slices and fairness

Model does not learn

- **Loss NaN:** lr too high, log(0), bad data; clip grads
- **Loss flat:** lr too low, dead ReLUs, frozen params, no `zero_grad`
- **Cannot overfit 1 batch:** bug in model / labels / loss
- Wrong loss input (softmax before CE), wrong shapes or dtype
- Labels shuffled apart from inputs; train/eval mode wrong
- Unnormalised inputs or targets; check data pipeline visually
- Val worse than train by far: overfitting, leakage or drift
- Compare with a trivial baseline and a tiny known-good model

Overfitting remedies

- More / better data, augmentation
- Weight decay (AdamW), dropout 0.1-0.5
- Smaller model, early stopping
- Label smoothing, mixup, stochastic depth
- Transfer learning / pretrained backbone
- Cross-validation for small tabular data
- Underfitting: bigger model, train longer, lower regularisation, better features

Architecture chooser

Data	Start with
Tabular	GBM first (XGBoost / LightGBM / CatBoost); MLP if huge
Images	CNN / ResNet, ViT with pretraining
Sequences, text	transformer; RNN / LSTM for small streaming cases
Graphs	GNN (message passing)
Generation	transformer LM, diffusion, VAE, GAN
Decisions	reinforcement learning
Few labels	pretrained / foundation model + fine-tune

Framework chooser

scikit-learn	classic ML, tabular, pipelines, baselines
PyTorch	research, custom models, LLMs (default)
TensorFlow / Keras 3	Keras API, TF Serving, LiteRT mobile
JAX	functional, TPU, large-scale research
GBM libs	XGBoost 3.4, LightGBM 4.7, CatBoost 1.2

Deployment formats

PyTorch	`torch.export` (.pt2), AOTInductor, ExecuTorch (.pte)
Portable	ONNX (.onnx) with ONNX Runtime
TensorFlow	SavedModel / `model.export` + TF Serving
Mobile / edge	LiteRT (.tflite), ExecuTorch
Weights only	safetensors, `.keras`, `state_dict`
Shrink	quantisation, pruning, distillation

Versions and notes

- Python 3.13; PyTorch 2.15; TensorFlow 2.21 + Keras 3.15
- scikit-learn 1.9; XGBoost 3.4; LightGBM 4.7; CatBoost 1.2; JAX 0.12
- Checked October 2026; verify against official docs
- Mixed precision: bf16 autocast on modern GPUs
- Track experiments (MLflow / W&B / TensorBoard)

Generated with AI assistance; verify against official documentation. See the Machine Learning section of the docs site for details.